Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Conditional Reinforcement Learning for Driving Policy

Semester Thesis

Xinyu Sun
Department of Mechanical and Processing Engineering

Advisors: Yan Wu
Supervisor: Prof. Dr. Fisher Yu

February 18, 2022

Abstract

In this work, we present a conditional reinforcement learning framework to learn driving policy from visual inputs, following high-level navigational commands. Involving high-level command control makes the autonomous driving system more applicable in the real environment. Also, Reinforcement Learning (RL) has recently demonstrated its efficacy in learning favorable policies from complex observations. To this end, we suggest a deep RL-based Conditional Predictive Control (CPC) framework to address the vision-based conditional driving task. CPC builds an Actor-Critic based policy network to select optimal actions in response to the current navigational command, and it forecasts the future semantic segmentation, speed, and collision event as auxiliary tasks to help with feature learning from action and observation. We implement experiments in CARLA, an interactive driving simulator, and our method shows competitiveness, even compared with other models with expert demonstration.

Acknowledgements

I give my heartfelt thanks to all the people who have ever helped me in this project. My sincere and hearty thanks and appreciations go firstly to my supervisors, Prof. Fisher Yu and Yan Wu, who provide a concrete proposal and useful suggestions and help me gain insight into autonomous driving and reinforcement learning. It is a great privilege and joy to study under their guidance and supervision. I am also extremely grateful to Jinkun Cao, who did the prior work and gave some practical advices before my work. Finally, I would express my appreciation to all friends who provided me assistance and support in the past few months.

Contents

1 Introduction **1**
1.1 Focus of this Work . . . 1
1.2 Thesis Organization . . . 2

2 Related Work **3**
2.1 Conditional Navigation . . . 3
2.2 End-to-End Reinforcement Learning . . . 3
2.3 Supervised Learning Affordance . . . 4
2.4 Controller . . . 4

3 Methods **5**
3.1 Problem Setup . . . 5
3.1.1 State and Action . . . 6
3.1.2 Reward and Objective . . . 6
3.2 Framework . . . 6
3.2.1 Prediction Network . . . 6
3.2.2 Actor and Critic Network . . . 7
3.2.3 Low-level Controller . . . 9

4 Experiments and Results **11**
4.1 Environment Setup . . . 11
4.1.1 Task Implement . . . 11
4.1.2 Reward Design . . . 12
4.2 Training Setup . . . 12
4.3 Method Comparison . . . 13
4.4 Ablation study . . . 14

5 Discussion **15**
5.1 Interpretability . . . 15
5.2 Limitation . . . 16

6 Conclusion **17**

List of Figures

3.1 Pipeline of CPC. 5
3.2 CPC framework. Given history observation and actions, the actor-network helps generate a policy and sample action sequences in action space. The prediction network perceives the current state and forecasts future states given action sequences. The critic network analyzes the future states and estimates the returns of the current state and action sequences. The critic network brings an explanation to the action decision. 7
3.3 Actor Network Structure. It generates a policy given current semantic segmentation and speed. The encoder and semantic layers are shared with Prediction Network and frozen when training the actor network. 8
3.4 Critic Network Structure. It predicted future returns (a scale) corresponding to features for each selected action sequences. 8
3.5 Local Lateral PID Controller. $\hat{p}$ is the target position under the original ego coordinates $\mathcal{C}$ after this short period with a fixed distance from the current position. e_0 is the original error between vehicle orientation and $\hat{p}$. e_i is a real-time error. 9

4.1 Three traffic situations in training 11
4.2 Left: how to track completion in NoCrash benchmark given a route. If the agent starts at w_0 and ends at w_2, the completion rate is $\frac{d_{01}+d_{12}}{d_{01}+d_{12}+d_{13}}$ Right: how to calculate the marching length l_t for a decision step t. $l_t = r_1 + d_{12} + r_2$. 12
4.3 The learning curves of methods under different command settings in CARLA town01 during exploration and exploitation. 13
4.4 The learning curves of CPC with additional $N-1$ prediction steps during exploration . . . 14
4.5 The learning curves of CPC under waypoint reward and continuous reward during exploration 14

5.1 Attribution points of visual inputs related to its policy under different high-level commands. 15
5.2 An Undesirable Driving Action under Red Traffic Light and the Reason 16

List of Tables

4.1 Successful rate comparison with driving policy learned by Reinforcement Learning (RL) in regular traffic 14

Chapter 1

Introduction

1.1 Focus of this Work

Autonomous driving is in the spotlight with some successful robotics applications of deep learning methods. We expect that an agent can drive safely with perceptual input while following high-level navigation commands. However, purely vision-based driving learning remains challenging due to strict reliability requirements and complex visual input. And it cannot respond to a navigational command or execute a specific turn at an impending intersection, which reduces its efficacy in real application.

Currently, most approaches [8, 19] to learning a driving policy are based on Imitation Learning (IL), where an agent learns to copy an expert's behavior. And IL methods integrate commands into their decision network as a gate, [9] proposed. IL possesses high data efficiency because each state(data) corresponds to a determinate action(label). IL agents can cope with lane-keeping and off-road avoidance while following navigational commands at intersections. However, the policy learned from expert demonstration suffers from serious data bias [10]. The expert demonstrations focus on only correct behaviors. Simple behaviors can deal with most situations while a heavy tail of complex reactions is required for rare events. Consequently, this performance possibly degrades with more data, if the rare events are neglected by the main mode of demonstrations.

On the contrary, RL method implements learning a driving policy by error-trial instead of telling agents what to do directly. The environment evaluates RL agents' performance by reward. In RL, agents are faced with more kinds of situations than IL agent in exploration, and it is supposed to understand how to avoid or recover from mistakes. However, training RL can need a magnitude larger amount of data than supervised learning to converge [25, 5]. Moreover, most RL relies on a replay buffer with capacity restriction. Therefore, most RL approaches are limited to scenarios with simple requirements, i.e. racing games [15], or simple inputs, i.e. kinetic states [7]. And many successful RL methods also [25, 7] need multi-stage training and expert demonstration to learn a reasonable policy. Moreover, the sufficient precondition of a good policy is still unclear. This is why most learning-based driving policies are black boxes. We don't know how agents make decisions.

Our prior work [17, 5] realized an explainable driving learning policy without expert's assistance. Their agents can forecast threats of action candidates without doing them. They addressed the data inefficiency by self-imitation learning, while clear the decision process by a rule-based controller and predicted safety-related event. However, there are two disadvantages. First, their agents cannot follow a specific guidance. Second, the rule-based controller will fail once their agents prophesy wrong or the rule doesn't apply to traffic scenarios. In this work, we aim to solve these two drawbacks. We use a conditional architecture

to follow high-level commands. We present an Actor-Critic RL agent for flexible action decision from its prediction. Our network can collect data and updates its policy iteratively without multi-stage demonstration. We summarize our main contributions below

- A conditional action selection and decision network to follow high-level commands.
- An RL agent online learning driving in the urban environment without expert demonstrations.
- An action-state Critic Network incorporating predicted affordances by a prediction network.
- A continuous reward design boosting agent's learning.

1.2 Thesis Organization

Our paper is organized as follows. Section 2 describes related work within the scope of autonomous driving based on RL. Section 3 explains the architecture of our driving decision system. Section 4 consists of the experiment setting, results, and ablation study. Section 5 discusses the interpretability and the limitation of our work. Section 6 is our conclusion.

Chapter 2

Related Work

2.1 Conditional Navigation

Guidance of robot navigation has been explored extensively [4]. A high level command is used as policy condition for agent, i.e. turning left, turning right and keeping straight. Commands can express our preference of policy to agent and resolve the ambiguity in favor of some course of action at intersections. [9] introduced two representation of commands to IL network, as a scalar input or as a gate of branch. As a scalar input, a command is embedded into a feature vector with high dimension, then command and vision features are concatenated and fed into following layers. As a gate, the command can select a branch among all branches to process corresponding vision feature. They showed the gate one outperformed the scalar input one. [7] introduced the command into their policy network as a final voter. Polices for all possible commands are inferred by the same network, but only the policy corresponding to the current command can output. The gate structure has been adapted by many IL and RL driving agents [8, 6, 13, 25]. We exploit the gate structure in our driving learning.

2.2 End-to-End Reinforcement Learning

RL agent collects its experience by error-trial and summarizes the law from its experience. The RL agent could learn how to recover from mistakes. Two basic approaches in RL are model-free and model-based.

In model-free RL, [13] utilized Actor-Critic architecture in its RL stage, showing the practicability of RL method. But they still need an expert in their IL stage. [25] manipulated value-based Rainbow-IQN Ape-X to handle complex tasks and won the Camera Only track of the CARLA challenge in 2020. But they also recruited an expert for data collecting. Besides, model-free approaches are less interpretable than model-based approaches. The model-based RL learns environment by estimating a transition function and a reward function and find the optimal policy by dynamic programming [3]. However, states are partially observable in autonomous driving. It will be overwhelmingly complex to optimize over such state-action spaces [22]. Some researchers reach a compromise between model-free RL and model-based RL. [7] learned expert's kinetic transition pattern, and updated its vision-based policy given the transition. It ranks top 10 in Carla leader board. In [17, 5], they use a ConvLstm network to predict the potential collision, offroad and offlane events for action evaluation.

2.3 Supervised Learning Affordance

A difficulty in training RL is that the sequence of observation encountered by an online RL agent is non-stationary [20, 16]. Therefore, the decision network updates are heavily correlated. Within the scope of autonomous driving, many researchers introduced supervised learning affordance to make agents aware of stationary and important components in traffic scenarios, which can help stabilize their decision networks. There are two different ways to adapt learning affordance in decision-making.

One is to use those affordances as input to a rule-based controller. [6] trained a network to obtain some decisive indicators, such as distance to the preceding car based on images, and then a controller interpreted the indicators into an action output based on a series of clear decision rules. [5, 17] trained a network to predict the potential collision or off-road events for current visual input and future actions. Then they added the weighted event possibility as a value and picked up the action with the least event possibility as output. These methods can explicitly show the decision-making theory. However, the controller is sensitive to the affordance inferred from the network. If the network is not robust or accurate enough in changing traffic scenarios, the controller can easily fail.

The other is to use those affordances as auxiliary tasks. The controller doesn't use the affordances straightforwardly, but the input of controller is provided by a supervised learning network. [14] exerted the affordances as a regularization norm in IL loss to increases transparency of the end-to-end network. [25] employs some perception auxiliary tasks to update the encoder in their model-free value-based RL agent. We follow our prior work [17, 5] affordance setting, but use the learning affordances as auxiliary tasks in our work.

2.4 Controller

The controller in an autonomous driving decision system can be divided into low-level controllers or middle-level controllers, according to its output representation. The output of a low-level controller can be directly used in interactions between agent and environment. In CARLA [11], [17, 5] produced low-level control signals in steer and throttle spaces, while [25, 7] had an extra brake control signal. The output of a middle-level controller must convert to low-level signals by an intermediary. [27] output a desirable curvature of driving trajectory and transferred it to steering control through Ackermann steering geometry in a real traffic situation. [6] mapped its affordances to a low-level controller by a designed linear function. [19] predicted the first 4 waypoints in future trajectory and employed PID controllers to interpret the waypoints as low-level control signals. We use the middle-level controller and a low-level PID controller to decouple steer, throttle, and brake control signals based on solid control theory.

Chapter 3

Methods

How can an agent learn driving interactively under a high-level command in multi-agent traffic scenarios? We design Conditional Reinforcement Learning for Predictive Control (CPC) to solve this problem. Figure 3.1 shows the pipeline of CPC. At each decision moment, CPC initiates with a real-time rgb image, speed signal and command. Then CPC selects some action candidates, forecasts the possible results after each action and picks up the action with the highest potential value after its comprehensive evaluation. Finally, a PID controller can convert the action to a low-level control signal for agent. Section 3.1 explains the formulation of problem. Section 3.2 shows the details of decision process. Action selection and evaluation are in subsection 3.2.2. Result forecasting is in subsection 3.2.1. Postprocessing from action to control signal is in subsection 3.2.3.

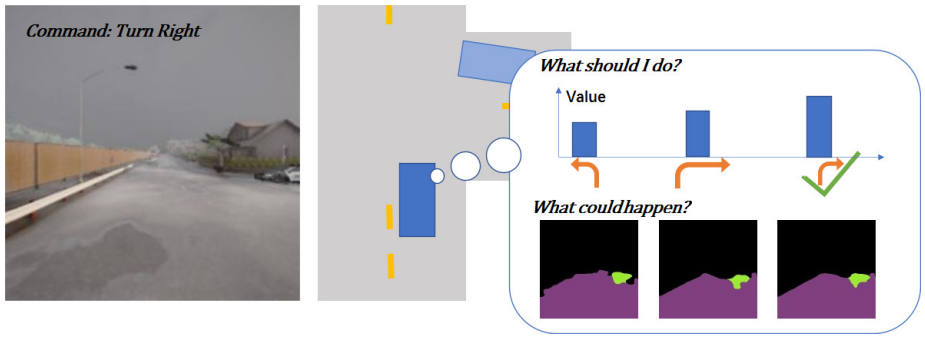

Figure 3.1: Pipeline of CPC.

3.1 Problem Setup

To implement conditional driving learning in multi-agent scenarios like the left part of figure 3.1, we formulate the problem with some assumptions as below.

3.1.1 State and Action

Following previous works, we assumed the environment transition follows a Markov Decision Process. With the state space S and the action space A, the transition is

$$s_{t+1} = M(s_t, a_t, a'_t) \tag{3.1}$$

where $a_t \sim \pi(s_t)$ is action executed by our ego agent at timestamp t, $a'_t \sim \pi'(s_t)$ are actions executed by other agents, s_t is the partial observation of the world from our ego agent's perspective. s_t consists of observation memory with a fixed length, current observation, and the current speed of the ego agent. $\pi'(s_t)$ is recommended by a traffic manager in the simulator and this function is supposed to be constant and unknown, so the transition function can be rewritten as

$$s_{t+1} = M(s_t, a_t) \tag{3.2}$$

In this work, action space A contains scaled planned position under ego coordinate $\mathcal{C}$ and speed at this position. The low-level controller are implemented by 3.2.3. A is continuous in 2D space $(-1, 1) \times (-1, 1)$, consisting of State space S consists of semantic segmentation $C \times W \times H$, collision event $\{0, 1\}$ and scaled real speed $(-1, 1)$. Following the prior work setting [5], $C = 6, W = H = 256$.

3.1.2 Reward and Objective

We only optimize the current ego vehicle's driving policy π. Unlike the previous works [5, 17], we pursue that an agent can follow high-level commands and finish a given route. The agent only receives high-level command c_t, and it is unaware of the route. Reward r is the projected safe-marching length of the route. Once the agent finishes a route, deviates, gets off the road, or crashes into others, the driving process will stop instantly, and it will turn to termination T. The objective of the optimization is to maximize the return R in the following N steps,

$$R(s_t, \pi) = \sum_{\tau=t}^{min(t+N,T)} r(s_\tau, a_\tau, c_\tau) \tag{3.3}$$

3.2 Framework

CPC is a decision-making system with state transition estimation to generate a middle-level controlling signal and convert it to low-level signals by two PID controllers. Similar to classic model-based RL, CPC includes a prediction network to predict the future state $N_a \times \tau \times |S|$, an actor to select N_a action sequence candidates $N_a \times \tau \times 2$ with potential maximal returns and a critic network to estimate the discounted returns $N_a \times 1$ of state and action within N steps.

3.2.1 Prediction Network

Full state prediction is too complex to realize because the simulator can only provide a partial observation of states to an ego vehicle and other agents' driving policy is unknown. First, we assume that other agents are part of the environment and that the ego agent can tell their behavior from the environment observation. Second, the state transition is independent with high-level commands directly, because the commands only participate in action selection and decision. Third, we assume that the environment state transition is coupled with the ego's state. Based on the assumptions above, we factorize the predicted state into three parts: ego

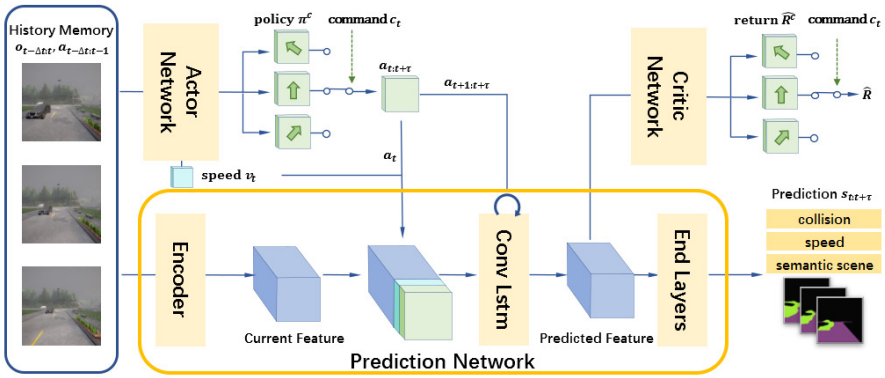

Figure 3.2: CPC framework. Given history observation and actions, the actor-network helps generate a policy and sample action sequences in action space. The prediction network perceives the current state and forecasts future states given action sequences. The critic network analyzes the future states and estimates the returns of the current state and action sequences. The critic network brings an explanation to the action decision.

speed, semantic segmentation, and collision event. Ego speed reflects the agent's own state independent of the environment. Semantic segmentation is the environment's state perceived from the RGB image, which can also reflect other agents' states. The collision event is an interaction between the environment and the ego agent, which can have an impact on the environment and agent state transition.

As shown in figure 3.2, the prediction network $\mathcal{T}$ utilizes an FPN encoder to process memorized RGB images and retain visual features. Then the high-level features are concatenated with sampled actions and current speed. Following [5], the combined features are fed into a convolutional LSTM [26] network to predict features in N timestamps. The predicted features pass through a speed layer, a semantic layer, and a collision layer to decode the predicted speed, semantic segmentation, and collision event in N timestamps correspondingly. These three layers' structures are similar.

The prediction network is supervised by 3.4. L_{speed} are MSE loss while $L_{semantic_{pred}}$ and $L_{collision}$ are $CrossEntropy$ loss. To boost the prediction training efficiency, the FPN encoder and the semantic layer are frozen during prediction network training. Their weights are only adjusted during current semantic segmentation training.

$$L_{prediction} = L_{speed} + L_{semantic_{pred}} + L_{collision} \tag{3.4}$$

3.2.2 Actor and Critic Network

Model-based RL can provide an explicit driving policy after learning an accurate transition model. However, it is time-consuming and hard to learn the transition exhaustively. Model-free RL can likely achieve an asymptotic performance without much exploration in worthless state spaces [23]. To trade off the learning efficiency and interpretability, we embed an actor-critic RL [18] into our framework and fine-tune it for action selection and decision. The actor network selects some action candidates given its policy distribution and the critic network picks up the action with the max estimated return.

The actor network $\mathcal{A}$ considers the ego and environment's states comprehensively and generates a gross control policy π. It grids each action dimension into three categories. At the first dimension, three categories represent stopping, running at a normal speed, and running at a high speed, whereas turning left, keeping straight, and turning right at the second dimension. $\pi(a|s)$ (hereinafter $\pi(a)$)represents the possibility of

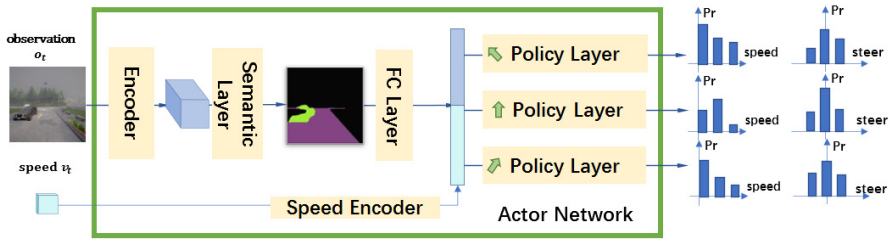

Figure 3.3: Actor Network Structure. It generates a policy given current semantic segmentation and speed. The encoder and semantic layers are shared with Prediction Network and frozen when training the actor network.

action a given the state s under current policy pi. As shown in figure 3.3, it borrows the encoder and semantic layer from the prediction network. It acquires a semantic segmentation of the current RGB image. The semantic segmentation turns into an environmental feature after postprocessing. The speed signal is encoded as an ego feature. Given a high-level command, the corresponding layer inferred a policy distribution from the combined feature. We assumed that π_{speed} and π_{steer} are conditionally independent. The cost of the actor network J_{actor} can be minimized as equation 3.5. We utilize advantage A to reduce variance and avoid overfitting [24]. The action sampling method follows the prior work [5]. A gross action is sampled given the actor's distribution. Then N_a action candidates are sampled uniformly within an interval corresponding to the gross action.

$$\begin{aligned} \nabla J_{actor} &= -(R(s,a) - A)\nabla \log \pi(a) \\ &= -(R(s,a) - A)\nabla(\log \pi_{speed}(a_{speed}) + \log \pi_{steer}(a_{steer})) \end{aligned} \tag{3.5}$$

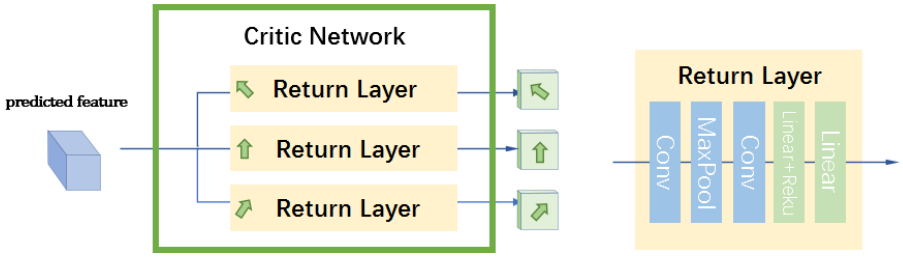

Figure 3.4: Critic Network Structure. It predicted future returns (a scale) corresponding to features for each selected action sequences.

The critic network $\mathcal{Q}$ estimates the return of the current state and action candidates according to the predicted feature. The intuition is that the evaluation of a driving policy is always result-oriented but not process-oriented. The transited state s_{t+1} is the result of (s, a). Besides, some situations are naturally dangerous, and they transit into termination state easily with few returns on average under most driving policies. Given the two reasons above, the average return can be inferred from the predicted feature when the prediction network $\mathcal{T}$ provides an accurate prediction about the ego and environment's states and their possible interaction after the transition. To incorporate all predicted information and avoid the bias of the handcrafted weights, the network $\mathcal{Q}$ adopts the predicted feature as input and maps it into a scale number by several convolution and linearized layers as figure 3.4 illustrates. The $\mathcal{Q}$ weights are updated by $SmoothL1$

loss J_Q to suppress some abnormal samples. The gradient is detached between network $\mathcal{T}$ and $\mathcal{Q}$.

$$\begin{aligned}\hat{R}(s_t, a_t) &= \mathcal{Q}(s_{t+1}) \\ &= \mathcal{Q}((\hat{s}_{t+1})) \\ &= \mathcal{Q}(\mathcal{T}(s_t, a_t))\end{aligned} \tag{3.6}$$

3.2.3 Low-level Controller

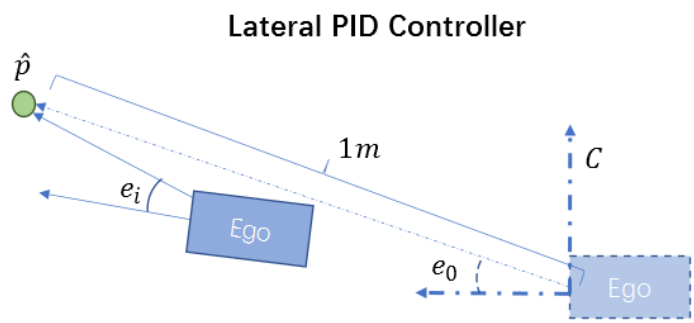

Figure 3.5: Local Lateral PID Controller. $\hat{p}$ is the target position under the original ego coordinates $\mathcal{C}$ after this short period with a fixed distance from the current position. e_0 is the original error between vehicle orientation and $\hat{p}$. e_i is a real-time error.

Like the classic model predictive control, CPC only implements a_t of the selected action sequence $a_{t:t+\tau}$ at timestamp t. First a_t are linear mapping to planned speed $\hat{v}$ and planned position $\hat{p}$ under $\mathcal{C}$. Then PID controllers 3.7 convert them to low-level values and run for a short period. In this short period, the longitudinal PID controller takes in throttle and brake to minimize the speed error whereas the lateral controller tracks the real-time position p_i under $\mathcal{C}$ and converts the position error to steering in figure 3.5. We adopt the default configuration as in the CARLA API [11].

$$output_i = K_p e_i + K_d \frac{de_i}{di} + K_i \int e_i di \tag{3.7}$$

Chapter 4

Experiments and Results

4.1 Environment Setup

The learning environment is CARLA 9.12 [11], which simulates variable traffic scenarios with built-in diverse scene structures, road configuration, and dynamic agents. The ego-vehicle uses a monocular RGB camera to get visual observations and a speed odometer to get real-time speed. The simulator runs on synchronization mode with a fixed frame rate. The time gap between the two frames is 0.1 seconds. CARLA supports extracting semantic segmentation and listening collision event for supervision and tracking real-time location for reinforcement learning reward design.

4.1.1 Task Implement

The objective of tasks is to train and test an agent who can follow the high-level command and drive safely under multiple scene structures. We refer to the settings of [10] as our training environment setting. We distribute conditional driving task to the ego agent suggested by [13] in three traffic situations, illustrated in figure 4.1, which can cover main traffic situations in Town01. Carla provides an API to generate some waypoints with continuous position and orientation given a resolution d_{res} a start position and an end by a global route planner, which can access the built-in map. The agent, unaware of the waypoints, can inquire the global route planner about an instant high-level command given its location.

Figure 4.1: Three traffic situations in training

We utilize a collision detector, an off-road detector, and a GNSS sensor to check whether the termination T condition meets. Once the agent runs out of roads or crashes with others in a decision step, i.e. vehicles and static objects, the current episode terminates. Besides, GNSS can tell whether the vehicle deviates from

the route. If the distance between the ego vehicle and any waypoint is over a threshold D, it stops and starts a new episode.

$$D = \begin{cases} 2\, d_{res}, c &= straight \\ 2.4\, d_{res}, c &= right \\ 3\, d_{res}, c &= left \end{cases} \tag{4.1}$$

4.1.2 Reward Design

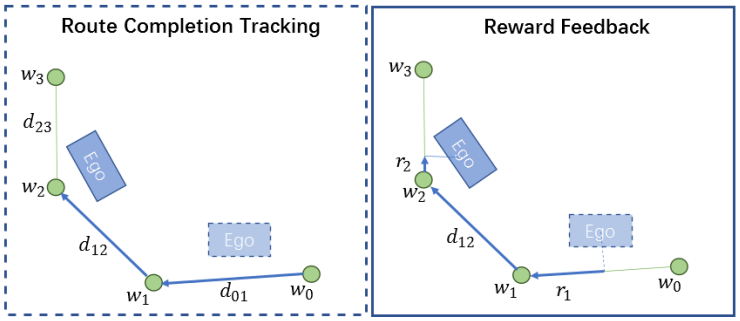

Figure 4.2: Left: how to track completion in NoCrash benchmark given a route. If the agent starts at w_0 and ends at w_2, the completion rate is $\frac{d_{01}+d_{12}}{d_{01}+d_{12}+d_{13}}$ Right: how to calculate the marching length l_t for a decision step t. $l_t = r_1 + d_{12} + r_2$.

Reward signals have an enormous impact on the learning efficiency of RL. Many RL methods suffer from sampling inefficiency due to sparse reward signals from the environment. We design a continuous reward signal for RL to reduce the latency of feedback. The intuition comes from the route completion rate in [10], where the rate relies on whether the agent chronologically passes through waypoints provided by CARLA API. The waypoints are invisible. Here we record the marching length l_t between decision steps t and $t+1$, and project this length on the direction from the current waypoint to the next waypoint. If the vehicle runs slowly and stays between two waypoints, it can still receive some encouragement from the environment. Besides, reward will deduct if an agent results in collision, offlane or offroad within a decision step.

$$r(s_t, a_t) = l_t - I(collision) - I(offroad) - 0.2I(offlane) \tag{4.2}$$

4.2 Training Setup

CPC grasp a driving policy by online learning algorithm 1 without extra data augmentation for multi-stage training. We adapt the ϵ greedy method [2] in data collection. The ego agent alternates its exploration policy per episode between its policy or ϵ greedy method at the end of an episode. In exploration, the ego agent samples N_a actions given actor's distribution, and then adopt the action with max return potential estimated by the critic network. The FPN encoder and semantic layer in T warm up with 2K images. The replay buffer stores data of 5000 steps at most. If it is full, new data will override old data.

Algorithm 1 Conditional Reinforcement Learning for Driving Policy

D: an empty data buffer
α: learning rate
$\mathcal{T}$, $\mathcal{A}$ and $\mathcal{Q}$: prediction, actor and critic network initialized with Kaiming weights [12] θ_T, θ_A and θ_Q
L_T: prediction loss
J_A, J_Q: cost function of actor and critic
while not visit N_{epi} trajectories **do**
    Collect trajectories d under random policy and CPC policy.
    $D \leftarrow D \cup d$
    Train $\mathcal{T}$: $\theta_T \leftarrow \theta_T - \alpha \nabla L_T$ until overfitting.
    Train $\mathcal{A}$: $\theta_A \leftarrow \theta_A - \alpha \nabla J_A$ until convergence.
    Train $\mathcal{Q}$: $\theta_Q \leftarrow \theta_Q - \alpha \nabla J_Q$ until convergence.
end while

4.3 Method Comparison

To evaluate the learning efficiency, we compare CPC with end-to-end learning strategies SPC [17] and DDPG [21] below. [17] represents self-imitative driving strategies with handcrafted value and [21] represents implicit policy-based driving strategies. From figure 4.3, our method has higher learning efficiency than other methods for conditional driving.

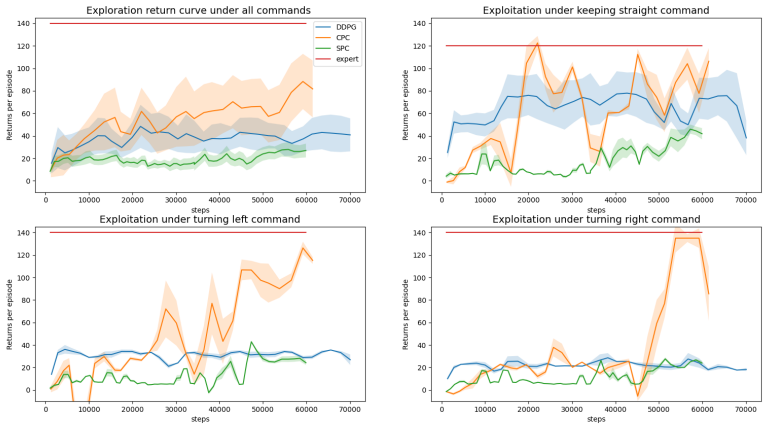

Figure 4.3: The learning curves of methods under different command settings in CARLA town01 during exploration and exploitation.

To evaluate the learned driving policy, we compare CPC with Rails [7]. Rails is leading in CARLA leaderboard among published RL methods based on camera and GNSS input. Rails is trained in multi-stages. It accesses the privileged map setting during training and processes Q learning of ego position according to the map. CPC is matched with other RL multistaged methods in the driving tasks table 4.1 without extra demonstration.

Table 4.1: Successful rate comparison with driving policy learned by RL in regular traffic

	Left	Straight	Right
Rails	100	100	100
CPC	80	100	80

4.4 Ablation study

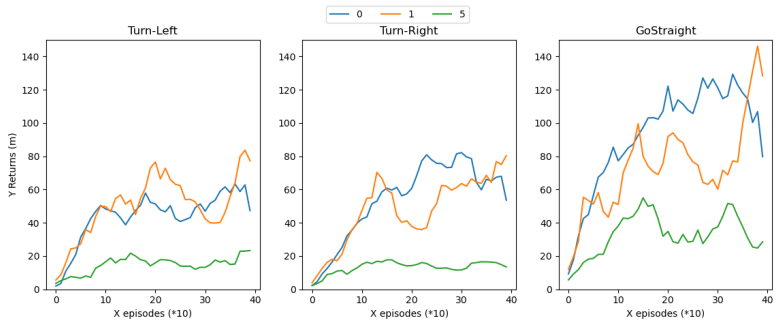

Figure 4.4: The learning curves of CPC with additional $N-1$ prediction steps during exploration

Actor Network $\mathcal{A}$ estimates an action distribution for visual input, providing cues for a gross middle-level control. Action sampling sequence selects action with better tolerance of unpredictable controlling noise, like road surface changes. An ablation study for the predicted action length is illustrated in figure 4.4. As the figure shows, a suitable predicted length N can improve the learning efficiency. The return increases slower with a longer predicted length. This is because the action distribution for the current frame doesn't fit the predicted future. For example, a turning can finish within fewer N steps, where N action in each step doesn't follow the identical distribution. Then the sampler failed to provide a reasonable action sequence when we increase the prediction length N. Therefore, the critic network cannot select a reasonable action from unreasonable candidates with a decreasing sampling efficiency.

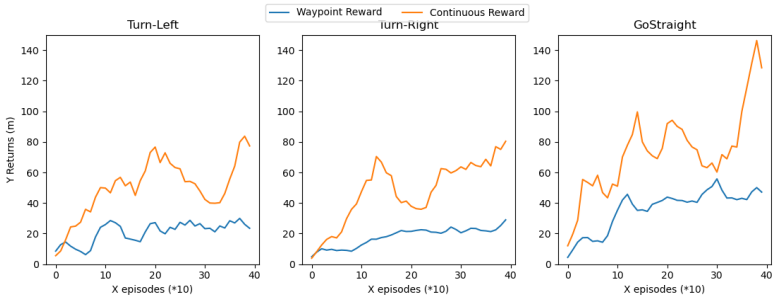

Figure 4.5: The learning curves of CPC under waypoint reward and continuous reward during exploration

Another ablation study for the reward is illustrated in figure 4.5. We compared the completion reward with continuous reward figure 4.2. We can see a continuous reward can provide feedback more instant and accurate, which can enhance exploration efficiency.

Chapter 5

Discussion

5.1 Interpretability

In our work, we develop a driving-learning strategy by reinforcement learning. The learned strategy can fuse visual inputs and physics measurement, extract a merged feature and recommend a policy given the inputs and high-level command. We adopt ZENNIT [1] to test the interpretability. As figure 5.1 illustrates, we can see which pixels mainly contribute to CPC decision. CPC focuses on the edge pixels of roads like human beings. When CPC properly understands its semantic surroundings, it can grasp key components in its visual inputs and infer a reasonable driving policy according to these key components instead of purely behavior clone. Compared to the previous work[5, 17], it doesn't require fine-tuned handcrafted weights for decision-making. Therefore, it has more flexibility to deal with the events unnoticed by rule designer.

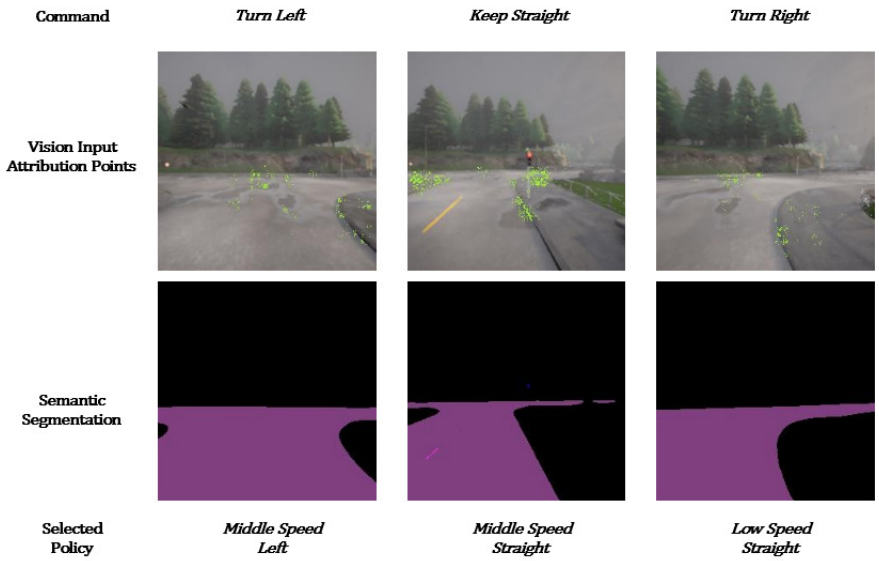

Figure 5.1: Attribution points of visual inputs related to its policy under different high-level commands.

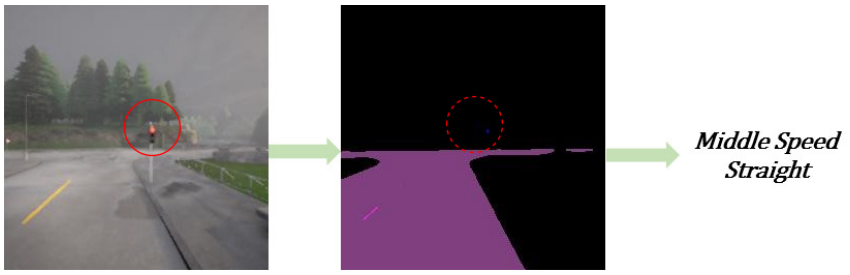

Figure 5.2: An Undesirable Driving Action under Red Traffic Light and the Reason

5.2 Limitation

There are mainly three limitations of CPC.

First, the detection of traffic lights and dealing with random pedestrians are not implemented yet. The actor network decides to speed up at turning because it cannot capture the red traffic light information from its input, as Figure 5.2 illustrates.

Second, action selections and decisions in CPC are heavily based on semantic segmentation. If the semantic encoder and decoder lost their efficacy to interpret visual input due to environmental change, i.e. dramatic illumination change, CPC will misunderstand its surroundings and possibly fail. This is the main reason why CPC fails in some dangerous situations. This problem can be solved by semantic data augmentation.

Third, CPC is an empirical RL method. Unlike systems with explicit decision strategy [5, 17], it naturally suffers from implicitness. Therefore, CPC lacks a solid mathematic foundation to prove its efficacy. But we can intuitively see the correlation between inputs and outputs from figure 5.1.

Chapter 6

Conclusion

In this work, we present a RL agent for end-to-end urban driving based using Actor-Critic algorithm with conditional network architecture. To smooth the interaction between agent and environment, we design a continuous reward for online training. To extract informative features from a challenging autonomous driving context, we incorporate transition modeling of model-based RL in the critic network as supervised prediction learning. We validate our design choices by ablation studies and compare our performance with other RL methods. We explore the causal relationship between inputs and the decision of the network. In future work, it can be refined by adding traffic light detection. It could be interesting to test our work in a real car.

Bibliography

[1] Christopher J. Anders, David Neumann, Wojciech Samek, Klaus-Robert Müller, and Sebastian Lapuschkin. Software for dataset-wide xai: From local explanations to global insights with Zennit, CoRelAy, and ViRelAy. *CoRR*, abs/2106.13200, 2021.

[2] Peter Auer, Nicolo Cesa-Bianchi, Yoav Freund, and Robert E Schapire. The nonstochastic multiarmed bandit problem. *SIAM journal on computing*, 32(1):48–77, 2002.

[3] Dimitri Bertsekas. *Dynamic programming and optimal control: Volume I*, volume 1. Athena scientific, 2012.

[4] Alexander Broad, Jacob Arkin, Nathan Ratliff, Thomas Howard, and Brenna Argall. Real-time natural language corrections for assistive robotic manipulators. *The International Journal of Robotics Research*, 36(5-7):684–698, 2017.

[5] Jinkun Cao, Xin Wang, Trevor Darrell, and Fisher Yu. Instance-aware predictive navigation in multi-agent environments. *arXiv preprint arXiv:2101.05893*, 2021.

[6] Chenyi Chen, Ari Seff, Alain Kornhauser, and Jianxiong Xiao. Deepdriving: Learning affordance for direct perception in autonomous driving. In *Proceedings of the IEEE international conference on computer vision*, pages 2722–2730, 2015.

[7] Dian Chen, Vladlen Koltun, and Philipp Krähenbühl. Learning to drive from a world on rails. *arXiv preprint arXiv:2105.00636*, 2021.

[8] Dian Chen, Brady Zhou, Vladlen Koltun, and Philipp Krähenbühl. Learning by cheating. In *Conference on Robot Learning*, pages 66–75. PMLR, 2020.

[9] Felipe Codevilla, Matthias Müller, Antonio López, Vladlen Koltun, and Alexey Dosovitskiy. End-to-end driving via conditional imitation learning. In *2018 IEEE International Conference on Robotics and Automation (ICRA)*, pages 4693–4700. IEEE, 2018.

[10] Felipe Codevilla, Eder Santana, Antonio M López, and Adrien Gaidon. Exploring the limitations of behavior cloning for autonomous driving. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 9329–9338, 2019.

[11] Alexey Dosovitskiy, German Ros, Felipe Codevilla, Antonio Lopez, and Vladlen Koltun. CARLA: An open urban driving simulator. In *Proceedings of the 1st Annual Conference on Robot Learning*, pages 1–16, 2017.

[12] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In *Proceedings of the IEEE international conference on computer vision*, pages 1026–1034, 2015.

[13] Xiaodan Liang, Tairui Wang, Luona Yang, and Eric Xing. Cirl: Controllable imitative reinforcement learning for vision-based self-driving. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 584–599, 2018.

[14] Ashish Mehta, Adithya Subramanian, and Anbumani Subramanian. Learning end-to-end autonomous driving using guided auxiliary supervision. In *Proceedings of the 11th Indian Conference on Computer Vision, Graphics and Image Processing*, pages 1–8, 2018.

[15] Volodymyr Mnih, Adria Puigdomenech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous methods for deep reinforcement learning. In *International conference on machine learning*, pages 1928–1937. PMLR, 2016.

[16] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller. Playing atari with deep reinforcement learning. *arXiv preprint arXiv:1312.5602*, 2013.

[17] Xinlei Pan, Xiangyu Chen, Qizhi Cai, John Canny, and Fisher Yu. Semantic predictive control for explainable and efficient policy learning. In *2019 International Conference on Robotics and Automation (ICRA)*, pages 3203–3209, 2019.

[18] Jan Peters and Stefan Schaal. Natural actor-critic. *Neurocomputing*, 71(7-9):1180–1190, 2008.

[19] Aditya Prakash, Kashyap Chitta, and Andreas Geiger. Multi-modal fusion transformer for end-to-end autonomous driving. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 7077–7087, 2021.

[20] Martin Riedmiller. Neural fitted q iteration–first experiences with a data efficient neural reinforcement learning method. In *European conference on machine learning*, pages 317–328. Springer, 2005.

[21] David Silver, Guy Lever, Nicolas Heess, Thomas Degris, Daan Wierstra, and Martin Riedmiller. Deterministic policy gradient algorithms. In *International conference on machine learning*, pages 387–395. PMLR, 2014.

[22] Richard D Smallwood and Edward J Sondik. The optimal control of partially observable markov processes over a finite horizon. *Operations research*, 21(5):1071–1088, 1973.

[23] Richard S Sutton and Andrew G Barto. *Reinforcement learning: An introduction*. MIT press, 2018.

[24] Richard S Sutton, David A McAllester, Satinder P Singh, and Yishay Mansour. Policy gradient methods for reinforcement learning with function approximation. In *Advances in neural information processing systems*, pages 1057–1063, 2000.

[25] Marin Toromanoff, Emilie Wirbel, and Fabien Moutarde. End-to-end model-free reinforcement learning for urban driving using implicit affordances. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2020.

[26] SHI Xingjian, Zhourong Chen, Hao Wang, Dit-Yan Yeung, Wai-Kin Wong, and Wang-chun Woo. Convolutional lstm network: A machine learning approach for precipitation nowcasting. In *Advances in neural information processing systems*, pages 802–810, 2015.

[27] Luona Yang, Xiaodan Liang, Tairui Wang, and Eric Xing. Real-to-virtual domain unification for end-to-end autonomous driving. In *Proceedings of the European conference on computer vision (ECCV)*, pages 530–545, 2018.